

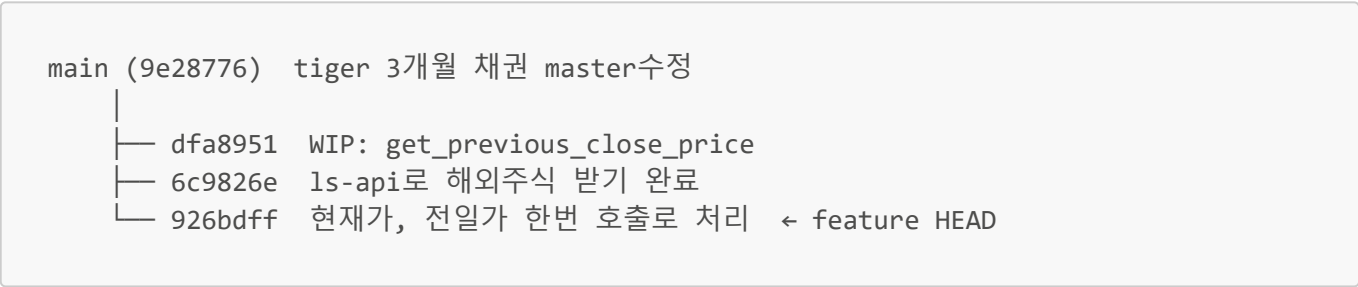
Git 병합 가이드: feature/migrate-yfinance-to-ls-api → main

작성일: 2026-05-29
프로젝트: StockTrader

1. 배경

- 작업 브랜치: feature/migrate-yfinance-to-ls-api
- 목표: yfinance 기반 해외 시세를 LS Open API(g3106 등)로 이전한 기능을 main에 반영
- Git Graph 확인 시: 분기 이후 main에는 새 커밋이 없고, feature 브랜치만 3커밋 앞서 있는 일직선 구조
→ Fast-forward 병합 가능

브랜치 관계 (병합 전 기준)



항목	값
main (병합 전)	9e28776 — tiger 3개월 채권 master수정
feature/migrate-yfinance-to-ls-api (병합 전)	926bdff — 현재가, 전일가 한번 호출로 처리
main 대비 추가 커밋	3개
충돌 가능성	낮음 (main 무변경 시 거의 없음)

브랜치 이름 주의: feature/imgrate-... 가 아니라 feature/migrate-yfinance-to-ls-api 입니다.

feature 브랜치에 포함된 주요 변경 (요약)

- LS API: ls_g3106.py, portfolio.py의 get_quote() / prefetch_quotes()
- 전일가·현재가 API 1회 조회 및 캐시
- get_company_name(): product_master.json 우선 (LS/yfinance 폴백 제거)
- 기타 Streamlit-TR(t2111) 관련 개선

2. 병합 유형: Fast-forward

main이 feature 분기 이후 움직이지 않았으므로:

```
git merge feature/migrate-yfinance-to-ls-api
```

- **Fast-forward** 로 **main** 포인터만 앞으로 이동합니다.
- 별도 merge commit 없이 feature의 커밋들이 **main** 히스토리에 그대로 이어집니다.

3. 방법 A — 로컬에서 main에 병합 (권장·단순)

PowerShell, 프로젝트 루트(E:\dev\StockTrader)에서 순서대로 실행합니다.

0단계: 작업 트리 정리

```
cd E:\dev\StockTrader
git status
```

- 수정·미추적 파일이 있으면 **커밋** 또는 **stash** 후 진행

```
git stash push -m "wip before merge" # 필요 시
```

1단계: feature 브랜치에서 동작 확인 (선택)

```
git checkout feature/migrate-yfinance-to-ls-api
# Streamlit / pytest 등으로 최종 확인
```

2단계: main으로 전환

```
git checkout main
```

3단계: 원격 main 동기화

```
git pull origin main
```

- **Already up to date** 이면 그대로 진행

4단계: feature 병합

```
git merge feature/migrate-yfinance-to-ls-api
```

기대 출력 예:

```
Updating 9e28776..926bdfb
Fast-forward
...
```

5단계: 원격 main에 push

```
git push origin main
```

6단계: feature 브랜치 정리 (선택)

```
git branch -d feature/migrate-yfinance-to-ls-api
git push origin --delete feature/migrate-yfinance-to-ls-api
```

4. 방법 B — GitHub Pull Request

리뷰·이력을 GitHub에 남기려면:

```
git push -u origin feature/migrate-yfinance-to-ls-api
```

GitHub에서:

1. **New Pull Request**
2. **base:** `main` ← **compare:** `feature/migrate-yfinance-to-ls-api`
3. 충돌 없음 확인 후 **Merge** (Fast-forward 또는 Create merge commit)

CLI (`gh` 설치 시):

```
gh pr create --base main --head feature/migrate-yfinance-to-ls-api --title
"Migrate overseas quotes to LS API" --body "...
gh pr merge --merge
```

5. 병합 후 확인

```
git log --oneline -5
git branch -v
git rev-parse main feature/migrate-yfinance-to-ls-api
```

확인 포인트:

- ☐ `main` HEAD = `926bdff` (또는 feature 최신 커밋)
- ☐ `git log main..feature` 가 비어 있음 (feature에만 있는 커밋 없음)
- ☐ Streamlit Cloud 배포 브랜치가 `main`이면 push 후 재배포 확인

6. 현재 저장소 상태 (문서 작성 시점)

로컬에서 확인한 결과:

브랜치	커밋 (short)	비고
<code>main</code> (현재 checkout)	<code>926bdff</code>	feature와 동일
<code>feature/migrate-yfinance-to-ls-api</code>	<code>926bdff</code>	병합 완료된 상태로 보임

→ 로컬 **fast-forward** 병합은 이미 완료된 것으로 보입니다.

→ 원격에 아직 반영하지 않았다면 `git push origin main` 만 실행하면 됩니다.

```
git push origin main
```

7. 문제 발생 시

충돌 (conflict)

`main`이 그 사이에 업데이트된 경우:

```
git merge feature/migrate-yfinance-to-ls-api
# 충돌 파일 수정 후
git add .
git commit -m "Merge feature/migrate-yfinance-to-ls-api into main"
git push origin main
```

잘못된 브랜치에서 작업한 경우

```
git branch -m feature/migrate-yfinance-to-ls-api # 이름 정정
```

병합 취소 (merge 직후, push 전)

```
git reset --hard ORIG_HEAD
```

`push` 이후 되돌리기는 `revert` 또는 팀 규칙에 맞는 방법을 사용하세요.

8. 한 줄 요약

```
checkout main → pull → merge feature/migrate-yfinance-to-ls-api → push origin main
```

`main`이 분기 후 변하지 않았으므로 **충돌 없이 Fast-forward** 가 정상 시나리오입니다.